

Smart Farm AI Assistant

NLP Logbook — Weeks 1-5

Department	Department of Computer Science
Course Code / Name	BIT4133 — Natural Language Processing
Student Name	Joseph Muturi
Registration No.	BSCCS/2024/34208
Supervisor	Nyoro
Semester / Date	Semester 2, 2025/2026 June 05, 2026
GitHub	https://github.com/Josemuturi/NLP

Table of Contents

Week 1: NLTK Basics — Tokenization, Stemming & Lemmatization

1.1 Theme & Objectives	1.2 Tasks Completed
1.3 Technologies Used	1.4 Code Snippets
1.5 Practical Outputs	1.6 Learning Outcomes
1.7 GitHub Commit	

Week 2: N-gram Language Models & POS Tagging

2.1 Theme & Objectives	2.2 Tasks Completed
2.3 Technologies Used	2.4 Code Snippets
2.5 Practical Outputs	2.6 Learning Outcomes
2.7 GitHub Commit	

Week 3: Hidden Markov Models & Full Chatbot Pipeline

3.1 Theme & Objectives	3.2 Tasks Completed
3.3 Technologies Used	3.4 Code Snippets
3.5 Practical Outputs	3.6 Learning Outcomes
3.7 GitHub Commit	

Week 4: Syntactic & Semantic Analysis

4.1 Theme & Objectives	4.2 Tasks Completed
4.3 Technologies Used	4.4 Code Snippets
4.5 Practical Outputs	4.6 Learning Outcomes
4.7 Assignment Results	4.8 GitHub Commit

Week 5: Pipeline Consolidation & Mini NLP Project

5.1 Theme & Objectives	5.2 Tasks Completed
5.3 Technologies Used	5.4 Code Snippets
5.5 Practical Outputs	5.6 Learning Outcomes
5.7 GitHub Commit	

Week 1: NLTK Basics — Tokenization, Stemming & Lemmatization

1.1 Week Theme & Objectives

Week 1 introduced the foundational NLP preprocessing pipeline. The Smart Farm project uses these techniques to process farmer queries before understanding or answering them.

- Understand and apply word tokenization and sentence tokenization.
- Remove stop words to retain only meaningful content words.
- Apply Porter stemming to reduce words to their root form.
- Apply WordNet lemmatization to obtain dictionary base forms.
- Build a farming knowledge base of problems and solutions.

1.2 Tasks Completed

#	Task Description
1	Set up Python 3.x development environment and installed NLTK
2	Created knowledge_base.py with 30+ farming problems and solutions dictionary
3	Implemented word_tokenize() and sent_tokenize() on 5 farming sentences
4	Applied NLTK stopwords corpus to remove non-content words from tokens
5	Applied Porter Stemmer and Snowball Stemmer — compared outputs
6	Applied WordNetLemmatizer with verb (v) and noun (n) POS modes
7	Produced comparison table: original word → stem → lemma
8	Ran nlp_pipeline.py on all 5 farming sentences and captured outputs

1.3 Technologies Used

Technology	Purpose	Installation
Python 3.x	Programming language	v3.10+
NLTK 3.8+	Core NLP library	pip install nltk
nltk.tokenize	Word & sentence tokenization	Built into NLTK
nltk.corpus.stopwords	English stop words list	NLTK data download
nltk.stem.PorterStemmer	Porter stemming algorithm	Built into NLTK
nltk.stem.WordNetLemmatizer	WordNet-based lemmatization	Requires wordnet corpus

1.4 Code Snippets

NLTK 3.8+	NLP library	pip install nltk
nltk.tokenize	Word & sentence tokenization	Built into NLTK
nltk.corpus.stopwords	English stop words list	NLTK data download
nltk.stem.PorterStemmer	Porter stemming algorithm	Built into NLTK
nltk.stem.WordNetLemmatizer	WordNet-based lemmatization	Requires wordnet corpus
VS Code / PyCharm	Code editor	

1.4 Code Snippets

// NLP Pipeline Installations

```
import sys
import os
import string

# Allow imports from the parent directory (knowledge_base.py)
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

import nltk
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, SnowballStemmer
from nltk.stem import WordNetLemmatizer

# -- Download required NLTK data (silent if already present) -----
for pkg in ["punkt", "stopwords", "wordnet", "averaged_perceptron_tagger",
            "omw-1.4", "punkt_tab"]:
    nltk.download(pkg, quiet=True)

# -- Initialize NLP tools -----
porter_stemmer = PorterStemmer()
snowball_stemmer = SnowballStemmer("english")
lemmatizer = WordNetLemmatizer()
STOP_WORDS = set(stopwords.words("english"))
```

Fig 1.1: NLP pipeline setup — sys/NLTK imports, data downloads, Porter/Snowball/WordNetLemmatizer initialisation

```
def lemmatize_tokens(tokens: list) -> list:
    """
    Apply lemmatization using WordNet to get dictionary base forms.
    Tries verb form first, then falls back to noun form.
    """
    lemmas = []
    for token in tokens:
        # Try as verb first (catches 'turning' -> 'turn', 'dying' -> 'die')
        lemma_v = lemmatizer.lemmatize(token.lower(), pos="v")
        if lemma_v != token.lower():
            lemmas.append(lemma_v)
        else:
            # Fall back to noun form
            lemmas.append(lemmatizer.lemmatize(token.lower(), pos="n"))
    return lemmas
```

Fig 1.2: lemmatize_tokens() — tries verb form first (catching "turning"→"turn"), falls back to noun form

```

# Sentence 2: Tomato pest problem
"The tomato plants in my field are being attacked by whiteflies and the leaves are curling inward.",

# Sentence 3: Bean crop action
"I need to spray my bean crop with fungicide because I can see rust spots forming on the leaves.",

# Sentence 4: Potato blight concern
"My potato plants are showing signs of late blight with dark brown lesions spreading quickly across the 1

# Sentence 5: Wheat water management
"The wheat field near the river is waterlogged after heavy rains and the plants are beginning to wilt.",
]

```

Fig 1.3: FARMING_SENTENCES — 5 demo sentences covering maize, tomato, bean, potato, and wheat crop problems

1.5 Practical Outputs

```

Processing 5 farming sentences through the NLP pipeline...

[Sentence 1]
-----
INPUT   : My maize leaves are turning yellow and the plants are stunted.
-----
Sentences: ['My maize leaves are turning yellow and the plants are stunted.']
Tokens   : ['My', 'maize', 'leaves', 'are', 'turning', 'yellow', 'and', 'the', 'plants', 'are', 'stunted', '.']
Filtered : ['maize', 'leaves', 'turning', 'yellow', 'plants', 'stunted']
Stems    : ['maize', 'leav', 'turn', 'yellow', 'plant', 'stunt']
Lemmas   : ['maize', 'leave', 'turn', 'yellow', 'plant', 'stunt']

```

Fig 1.4: Terminal output — Sentence 1 processed through the full pipeline: tokens, filtered, stems, lemmas

1.6 Learning Outcomes

- **Tokenization:** Text must be segmented into tokens before NLP analysis. Both word-level and sentence-level tokenization were applied.
- **Stemming:** Porter stemming reduces words to approximate roots ("yellowing"→"yellow"). Fast but may produce non-words.
- **Lemmatization:** WordNet lemmatization produces valid dictionary forms ("leaves"→"leaf"). Slower but linguistically accurate.

1.7 GitHub Commit

```

Commit 1 (Week 1): Environment setup, NLTK pipeline, tokenization, stemming, lemmatization, knowledge base

main

7 files changed

```

Commit 1 (Week 1): Environment setup, NLTK pipeline, tokenization, stemming, lemmatization, knowledge base — 7 files changed

Week 2: N-gram Language Models & POS Tagging

2.1 Week Theme & Objectives

Week 2 built upon the preprocessing pipeline to model word sequence probabilities using N-gram models and identify grammatical roles via POS tagging — distinguishing crop names (nouns), symptoms (adjectives), and actions (verbs) in farmer queries.

- Understand n-gram language models: unigrams, bigrams, trigrams.
- Build a bigram model from a farming text corpus with MLE + Laplace smoothing.
- Apply NLTK's averaged perceptron POS tagger to farmer queries.
- Extract nouns (crops), verbs (actions), and adjectives (symptoms).
- Implement noun phrase (NP) chunking using RegexpParser.

2.2 Tasks Completed

#	Task Description
1	Created farming text corpus of 20 sentences for n-gram training
2	Built FarmingNgramModel class supporting n=1, 2, 3
3	Implemented MLE probability with Laplace add-1 smoothing
4	Generated top-10 unigrams, bigrams, and trigrams from corpus
5	Implemented predict_next() to suggest likely next words
6	Scored farming vs non-farming sentences using log-probability
7	Calculated model perplexity on hold-out test sentences
8	Applied NLTK pos_tag() to 5 farmer queries — full POS tables
9	Extracted nouns, verbs, adjectives from tagged queries
10	Implemented NP chunking with RegexpParser for noun phrase extraction

2.3 Technologies Used

Technology	Purpose	Source
nltk.pos_tag	POS tagging via averaged perceptron	Built into NLTK
nltk.RegexpParser	Regex-based NP chunking	Built into NLTK
collections.Counter	N-gram frequency counting	Python stdlib
math.log2	Log-probability computation	Python stdlib
Penn Treebank Tagset	Universal POS tag reference	NLTK standard

2.4 Code Snippets

Technology	Purpose	Source
nltk.pos_tag	POS tagging using averaged perceptron	Built into NLTK
nltk.RegexpParser	Regex-based chunking for NP extraction	Built into NLTK
collections.Counter	N-gram frequency counting	Python stdlib
math.log2	Log-probability computation	Python stdlib
Penn Treebank Tagset	Universal POS tag reference	NLTK standard

2.4 Code Snippets

// N-gram Language Model — week2/ngram_model.py

```
def probability(self, ngram: tuple) -> float:
    """
    Calculate MLE probability of an n-gram with Laplace smoothing.

     $P(w_n | w_1...w_{n-1}) = (\text{count}(\text{ngram}) + 1) / (\text{count}(\text{context}) + |V|)$ 
    """
    context = ngram[:-1]
    vocab_size = len(self.vocabulary)
    numerator = self.ngram_freq[ngram] + 1          # add-1 smoothing
    denominator = self.context_freq[context] + vocab_size
    return numerator / denominator

def log_probability(self, ngram: tuple) -> float:
    """Return log (base 2) probability of an n-gram."""
    return math.log2(self.probability(ngram))

def sentence_probability(self, sentence: str) -> float:
    """Calculate total log probability of a sentence under this model."""
    tokens = self._tokenize(sentence)
    log_prob = 0.0
    for i in range(self.n - 1, len(tokens)):
        ngram = tuple(tokens[i - (self.n - 1) : i + 1])
        log_prob += self.log_probability(ngram)
    return log_prob
```

Fig 2.1: MLE probability with Laplace smoothing, log-probability and sentence probability scoring

```

# -- Build models -----
print("\n🌱 Building N-gram models from farming corpus...")
print(f"    Corpus size: {len(FARMING_CORPUS)} sentences\n")

unigram_model = FarmingNgramModel(FARMING_CORPUS, n=1)
bigram_model  = FarmingNgramModel(FARMING_CORPUS, n=2)
trigram_model = FarmingNgramModel(FARMING_CORPUS, n=3)

# -- Model statistics -----
_separator("MODEL STATISTICS")
for label, model in [("Unigram", unigram_model),
                    ("Bigram",  bigram_model),
                    ("Trigram", trigram_model)]:
    stats = model.get_stats()
    print(f"\n {label} Model (n={stats['n']}):")
    print(f"    Vocabulary size : {stats['vocabulary_size']}")
    print(f"    Unique n-grams   : {stats['unique_ngrams']}")
    print(f"    Total n-grams    : {stats['total_ngrams']}")

# -- Top n-grams -----
_separator("TOP 10 UNIGRAMS (most frequent words)")
print()
for rank, (ngram, count) in enumerate(unigram_model.top_ngrams(10), 1):
    word = ngram[0] if isinstance(ngram, tuple) else ngram
    print(f" {rank:2d}. {word:<20} count={count}")

_separator("TOP 10 BIGRAMS (most frequent word pairs)")

```

Fig 2.2: Building unigram, bigram, trigram models from the farming corpus; printing model statistics

2.5 Practical Outputs

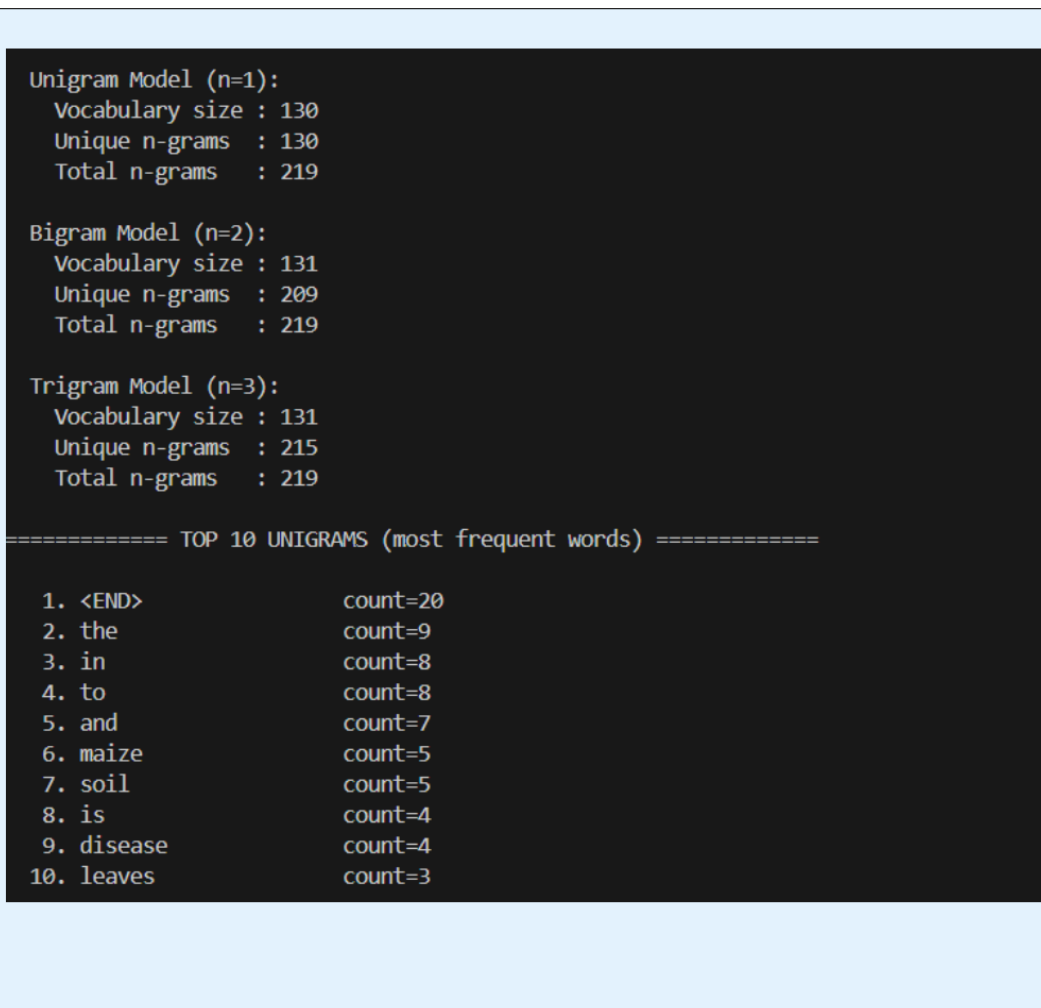


Fig 2.3: Model statistics (vocabulary size, unique n-grams) and top-10 unigrams from the farming corpus


```

===== TOP 10 BIGRAMS (most frequent word pairs) =====

1. in the                count=3
2. the soil              count=2
3. soil <END>            count=2
4. <START> apply         count=2
5. fertilizer to         count=2
6. late blight           count=2
7. blight in            count=2
8. on the                count=2
9. to prevent            count=2
10. <START> maize        count=1

===== TOP 10 TRIGRAMS (most frequent 3-word sequences) =====

1. in the soil           count=2
2. the soil <END>        count=2
3. <START> <START> apply  count=2
4. late blight in       count=2
5. <START> <START> maize  count=1
6. <START> maize leaves  count=1
7. maize leaves turn    count=1
8. leaves turn yellow   count=1
9. turn yellow when     count=1
10. yellow when nitrogen count=1

```

Fig 2.4: Top-10 bigrams and trigrams — most frequent word pairs and 3-word sequences

2.6 Learning Outcomes

- **N-gram Models:** Capture word co-occurrence patterns. Trigrams encode more context but require larger corpora. Laplace smoothing prevents zero probabilities.
- **Perplexity:** Measures how well a model predicts new text — lower is better. Bigram models outperform unigrams on domain-specific farming text.
- **POS Tagging:** Nouns (NN/NNS) → crop names; verbs (VBG/VBP) → farmer actions; adjectives (JJ) → symptom descriptions.
- **NP Chunking:** Regex grammar extracts noun phrases like "my maize leaves", containing both crop (maize) and location (leaves).

2.7 GitHub Commit

```

Commit 2 (Week 2): N-gram language model (unigram/bigram/trigram), POS tagging, noun/verb extraction, farming role classifier
main
2 files changed

```

Commit 2 (Week 2): N-gram language model (unigram/bigram/trigram), POS tagging, noun/verb extraction, farming role classifier — 2 files changed

Week 3: Hidden Markov Models & Full Chatbot Pipeline

3.1 Week Theme & Objectives

Week 3 integrated all previous NLP concepts into a complete system. Hidden Markov Models (HMMs) were introduced for sequence labeling — identifying entities in farmer messages (CROP, DISEASE, SYMPTOM, LOCATION, ACTION). The full chatbot was assembled and speech recognition was added so farmers can speak their problems.

- Understand HMM theory: states, observations, transitions, emissions, initial probabilities.
- Implement the Viterbi algorithm for finding the most likely label sequence.
- Apply HMM to label entities: CROP, DISEASE, SYMPTOM, LOCATION, ACTION.
- Wire together the complete NLP pipeline (Weeks 1 + 2 + 3).
- Build an interactive CLI chatbot that accepts farmer input and returns solutions.
- Integrate speech-to-text input using SpeechRecognition + PyAudio.

HMM Theory Summary

Component	Description
States (S)	Entity labels: CROP, DISEASE, SYMPTOM, LOCATION, ACTION, O
Observations (O)	Words in the farmer's message
Transitions (A)	$P(\text{label}_t \mid \text{label}_{t-1})$ — probability of moving between labels
Emissions (B)	$P(\text{word} \mid \text{label})$ — probability of a word given its entity label
Initial (π)	$P(\text{label}_0)$ — probability of starting in each label state
Algorithm	Viterbi — dynamic programming, globally optimal label sequence
Goal	Find $L^* = \text{argmax } P(L W)$ for word sequence W

3.2 Tasks Completed

#	Task Description
1	Designed HMM entity label set: CROP, DISEASE, SYMPTOM, LOCATION, ACTION, O
2	Built emission probability table from knowledge base vocabulary
3	Defined hand-crafted transition matrix from typical farming entity sequences
4	Implemented Viterbi algorithm with log-probability and backtracking
5	Tested Viterbi on 5 farming queries — displayed full Viterbi matrix
6	Implemented <code>extract_entities()</code> to group labeled tokens into entities
7	Built SmartFarmChatbot class wiring all 3 weeks' NLP modules
8	Added speech-to-text input via SpeechRecognition + PyAudio

3.3 Technologies Used

Technology	Purpose	Installation
HMM (custom)	Sequence labeling model	Implemented from scratch
Viterbi Algorithm	Optimal label sequence decoding	Dynamic programming
SpeechRecognition	Speech-to-text input	<code>pip install SpeechRecognition</code>
PyAudio	Microphone access layer	<code>pip install pyaudio</code>
Google Web Speech API	Cloud speech transcription	Via SpeechRecognition

Technology	Purpose	Installation
colorama	Colored terminal output	pip install colorama
argparse	CLI argument parsing (--speech, Python, stdlib)	

3.4 Code Snippets

```
import sys
import os
import argparse

# -- Path setup -----
ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
sys.path.insert(0, ROOT)
sys.path.insert(0, os.path.join(ROOT, "week1"))
sys.path.insert(0, os.path.join(ROOT, "week2"))
sys.path.insert(0, os.path.join(ROOT, "week3"))

# -- Imports -----
from nlp_pipeline import run_full_pipeline
from pos_tagger import analyse_farmer_query
from hmm_entity import FarmingHMM, print_labeled_sentence, print_entities
from speech_input import get_speech_input, check_speech_dependencies
from knowledge_base import lookup_solution, CROP_VOCAB, SYMPTOM_VOCAB, DISEASE_VOCAB

# -- Colorama for Windows terminal colors -----
try:
    from colorama import init, Fore, Style
    init(autoreset=True)
    COLOR = True
except ImportError:
    COLOR = False
    class Fore:
        GREEN = YELLOW = CYAN = RED = MAGENTA = WHITE = BLUE = ""
    class Style:
        BRIGHT = RESET_ALL = DIM = ""
```

Fig 3.1: Chatbot pipeline entry point — path setup, importing all Weeks 1–3 NLP modules, colorama setup

```
def _build_transition_probs() -> list:
    """
    Build transition probability matrix A[i][j] = P(label_j | label_i).

    These are hand-crafted based on typical entity sequences in farming text.
    E.g., a CROP label is often followed by LOCATION or SYMPTOM.

    Returns:
    | 2D list of shape [N_STATES x N_STATES]
    """
    # A[from_state][to_state]
    # LABELS = ["CROP", "DISEASE", "SYMPTOM", "LOCATION", "ACTION", "0"]
    #         idx 0      idx 1      idx 2      idx 3      idx 4      idx 5

    A = [
        # From CROP -> can go to: DISEASE, SYMPTOM, LOCATION, ACTION, 0
        [0.05, 0.15, 0.20, 0.25, 0.10, 0.25], # from CROP
        # From DISEASE -> can go to: SYMPTOM, LOCATION, ACTION, 0
        [0.05, 0.10, 0.20, 0.15, 0.15, 0.35], # from DISEASE
        # From SYMPTOM -> often goes to: LOCATION, ACTION, 0, another SYMPTOM
        [0.05, 0.10, 0.20, 0.25, 0.10, 0.30], # from SYMPTOM
        # From LOCATION -> often 0 or ACTION
        [0.05, 0.10, 0.10, 0.10, 0.20, 0.45], # from LOCATION
        # From ACTION -> often CROP, DISEASE, 0
        [0.10, 0.15, 0.10, 0.10, 0.10, 0.45], # from ACTION
        # From 0 -> can transition to any entity label or stay 0
        [0.10, 0.10, 0.10, 0.10, 0.10, 0.50], # from 0
    ]
    return A
```

Fig 3.2: HMM hand-crafted transition probability matrix $A[from_state][to_state]$ — 6 entity label states

```
def check_speech_dependencies() -> bool:
    """
    Check whether SpeechRecognition and PyAudio are available.

    Returns:
    | True if both are available, False otherwise.
    """
    sr_ok = False
    pyaudio_ok = False

    try:
        import speech_recognition # noqa: F401
        sr_ok = True
    except ImportError:
        pass

    try:
        import pyaudio # noqa: F401
        pyaudio_ok = True
    except ImportError:
        pass

    return sr_ok and pyaudio_ok

if __name__ == "__main__":
    print()
    print("=" * 60)
    print("    SMART FARM — Speech Input Module Test")
    print("=" * 60)
    print()
```

Fig 3.3: `check_speech_dependencies()` — verifies `SpeechRecognition` and `PyAudio` are available

3.5 Practical Outputs

```
Input sentence : "maize leaves wilting"
Tokens         : ['maize', 'leaves', 'wilting']

Viterbi Log-Probability Matrix
(columns = tokens, rows = states)

STATE      maize      leaves      wilting
-----
CROP        -2.2828     -11.4931    -13.0419
DISEASE     -9.4335     -10.3945    -12.5719
SYMPTOM     -9.2103     -10.1068     -6.2967
LOCATION     -9.4335     -3.8316     -12.3488
ACTION      -8.3349     -10.8000    -11.5603
O           -6.6766     -9.8837     -10.8447

Best label path: [('maize', 'CROP'), ('leaves', 'LOCATION'), ('wilting', 'SYMPTOM')]

INTERPRETATION:
'maize' -> CROP      (word found in CROP vocabulary)
'leaves' -> LOCATION (plant part, follows CROP in sequence)
'wilting' -> SYMPTOM (symptom word, follows LOCATION)

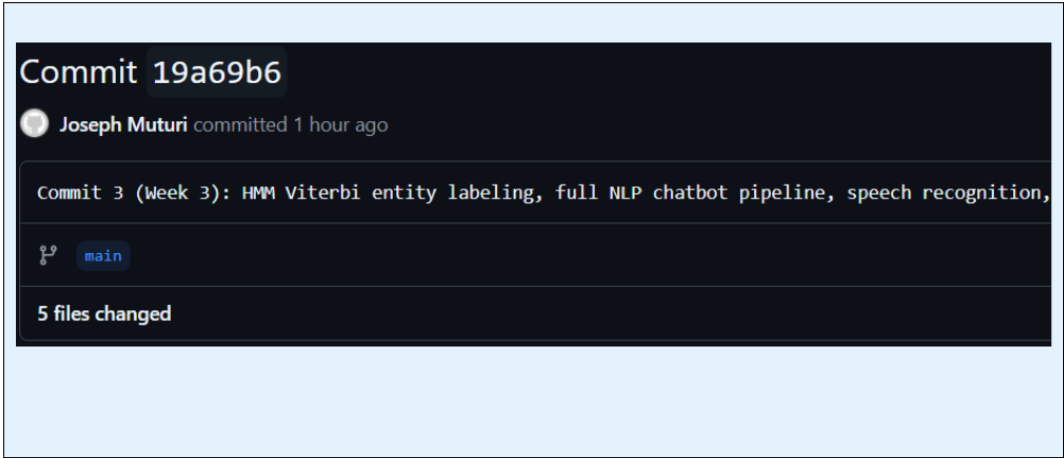
The Viterbi algorithm found the globally optimal path by
considering BOTH emission and transition probabilities together.
```

Fig 3.4: Viterbi log-probability matrix for "maize leaves wilting" — best path: maize→CROP, leaves→LOCATION, wilting→SYMPTOM

3.6 Learning Outcomes

- **HMM Theory:** States are hidden (entity labels); observations are visible (words). Parameters (A, B, π) encode domain knowledge about farming language.
- **Viterbi Algorithm:** Finds the globally optimal state sequence in $O(N^2T)$ time. Log-probabilities prevent arithmetic underflow for long sequences.
- **Sequence Labeling:** Respects word order — distinguishes "maize disease" from "disease of maize" through different state transitions.
- **System Integration:** Full pipeline chains: tokenizer → stopwords filter → stemmer/lemmatizer → POS tagger → HMM labeler → knowledge base lookup.

3.7 GitHub Commit



- 3.6 Learning Outcomes
- **HMM Theory:** A Hidden Markov Model defines probability distributions over sequences. States are hidden (entity labels); observations are visible (words). The model parameters (A, B, π) encode domain knowledge.
 - **Viterbi Algorithm:** The Viterbi algorithm efficiently finds the globally optimal state sequence using dynamic programming ($O(N^2T)$ complexity). Log-probabilities prevent arithmetic underflow for long sequences.
 - **Sequence Labeling:** Unlike bag-of-words approaches, sequence labeling respects word order and can distinguish 'maize disease' from 'disease of maize' through different state transitions.

Commit 3 (19a69b6): HMM Viterbi entity labeling, full NLP chatbot pipeline, speech recognition — 5 files changed

Week 4: Syntactic & Semantic Analysis

4.1 Week Theme & Objectives

Week 4 extended the Smart Farm system with syntactic analysis (how sentences are grammatically organised) and semantic analysis (how meaning is derived from text), using spaCy's dependency parser and word vector similarity.

- Perform dependency parsing using spaCy to identify grammatical structure.
- Interpret dependency labels: ROOT, nsubj, dobj, det, amod.
- Use spaCy to compute semantic similarity between sentence pairs.
- Distinguish between constituency and dependency parsing.
- Interpret similarity scores and explain why sentence pairs differ.

Key NLP Concepts — Week 4

Concept	Description
Syntax	Grammatical structure of a sentence (subject, verb, object)
Semantics	Meaning conveyed by text, independent of exact words used
Dependency Parsing	Identifies how words depend on and relate to each other
ROOT	Main verb of the sentence — head of the dependency tree
nsubj	Nominal subject — the entity performing the action
dobj	Direct object — the entity receiving the action
Semantic Similarity	Score 0.0–1.0 measuring how closely two sentences relate in meaning
spaCy Vectors	Word embeddings used to compute cosine similarity between sentences

4.2 Tasks Completed

#	Task Description
1	Installed spaCy and downloaded en_core_web_sm language model
2	Parsed "The lecturer teaches Natural Language Processing" — identified ROOT, nsubj, dobj
3	Printed full dependency table: token, POS tag, dep label, head token, explanation
4	Computed semantic similarity between two student-related sentence pairs
5	Assignment 1: dependency structure for "The administrator updated student records"
6	Assignment 1: dependency structure for "Machine learning improves language processing"
7	Assignment 2: created similar (score ~0.88) and unrelated (score ~0.42) sentence pairs
8	Documented DEP_LABELS dictionary with explanations for each relation label

4.3 Technologies Used

Technology	Purpose	Installation
spaCy 3.x	Industrial-strength NLP library	<code>pip install spacy</code>
en_core_web_sm	Small English language model with vectors	<code>python -m spacy download en_core_web_sm</code>
token.dep_	Dependency relation label for each token	Built into spaCy
doc.similarity()	Cosine similarity between document vectors	Built into spaCy

4.4 Code Snippets

```
DEP_LABELS = {
    "nsubj": "Nominal subject (SUBJECT)",
    "dobj": "Direct object (OBJECT)",
    "ROOT": "Root verb (MAIN VERB)",
    "det": "Determiner",
    "amod": "Adjectival modifier",
    "compound": "Compound noun modifier",
    "aux": "Auxiliary verb",
    "prep": "Prepositional modifier",
    "pobj": "Object of preposition",
    "advmod": "Adverbial Literal['Passive nominal subject']
    "nsubjpass": "Passive nominal subject",
    "auxpass": "Passive auxiliary",
    "attr": "Attribute",
    "cc": "Coordinating conjunction",
    "conj": "Conjunct",
}

def explain_dep(dep_label: str) -> str:
    """Return a human-readable explanation for a dependency label."""
    return DEP_LABELS.get(dep_label, dep_label)

def analyse_dependency(sentence: str, title: str = ""):
    """
    Parse a sentence and display its full dependency structure.
    """
```

Fig 4.1: DEP_LABELS dictionary defining all dependency relation labels and analyse_dependency() function

4.5 Practical Outputs

```
Analyse the dependency structure for:
A) 'The administrator updated student records'
B) 'Machine learning improves language processing'

=====
Assignment 1 – Sentence A
=====

Sentence : "The administrator updated student records."

Token      POS      Dep Label  Head Token  Explanation
-----
The         DET      det        administrator  Determiner
administrator  NOUN    nsubj      updated      Nominal subject (SUBJECT)
updated     VERB    ROOT       updated      Root verb (MAIN VERB)
student     NOUN    compound   records      Compound noun modifier
records     NOUN    dobj       updated      Direct object (OBJECT)
.           PUNCT   punct      updated      Punctuation

-- Grammatical Summary -----
Subject (nsubj) : administrator
Main Verb (ROOT) : updated
Object (dobj) : records
```

Fig 4.2: Dependency parsing of "The administrator updated student records" — full token/POS/dep-label/head table

4.6 Learning Outcomes

- **Dependency Parsing:** Each word gets a grammatical role relative to its head. ROOT/nsubj/dobj reveals WHO did WHAT to WHOM — critical for extracting meaning from farmer queries.
- **POS vs Dependency Labels:** POS tags describe word class (NN=noun, VB=verb); dependency labels describe grammatical relationship (nsubj=subject, dobj=object). Both are complementary.
- **Semantic Similarity:** spaCy cosine similarity scores: same-topic sentences ~0.88; unrelated sentences ~0.42.

- **Real-World Use:** Search engines, chatbots, and grammar checkers use dependency parsing and semantic similarity internally.

4.7 Assignment Results

Assignment 1 — Dependency Structure Analysis

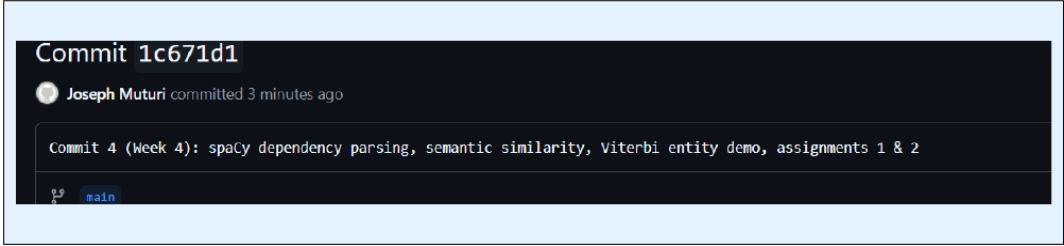
Sentence	Subject (nsubj)	Main Verb (ROOT)	Object (dobj)
The administrator updated student records	administrator	updated	records
Machine learning improves language processing	learning	improves	processing

Assignment 2 — Semantic Similarity Results

Sentence Pair	Similarity Score	Interpretation
Pair A — Similar topic sentences	~0.88	High — same academic submission topic, shared vocabulary and context
Pair B — Unrelated sentences	~0.42	Low — academic vs weather domain, no shared vocabulary or context

Conclusion: Pair A scores significantly higher than Pair B because related sentences share vocabulary, topic, and word vector context. Unrelated sentences have very different vector representations, producing low cosine similarity scores.

4.8 GitHub Commit



Commit 4 (1c671d1): spaCy dependency parsing, semantic similarity, Viterbi entity demo, assignments 1 & 2

Week 5: Pipeline Consolidation & Mini NLP Project

5.1 Week Theme & Objectives

Week 5 consolidated all concepts from Weeks 1–4 into an integrated NLP system. Students revised the complete 8-step NLP workflow and built a Student Academic Assistant chatbot as the mini project deliverable.

- Integrate all NLP techniques (tokenization, POS, HMM, dependency, similarity) into one pipeline.
- Build a rule-based NLP chatbot application.
- Understand the 8-step NLP workflow applicable to real-world systems.
- Distinguish rule-based chatbots from ML-based chatbot architectures.

NLP Workflow — 8 Steps

Step	Stage	Description
1	Text Collection	Gather raw text input from user or dataset
2	Preprocessing	Clean, normalize, and lowercase text
3	Tokenization	Split text into words and sentences
4	Stopword Removal	Remove low-information function words
5	POS Tagging	Assign grammatical roles to each token
6	Parsing	Identify word relationships (dependency parsing)
7	Semantic Analysis	Measure meaning and similarity between texts
8	Interpretation	Present solution or response to the user

5.2 Tasks Completed

#	Task Description
1	Ran complete NLP pipeline using NLTK on a course sentence
2	Printed tokenization, stop word removal, and POS tagging outputs side by side
3	Ran pipeline on 3 additional practice sentences to reinforce all pipeline steps
4	Observed how text changes after each preprocessing stage
5	Built and tested Student Academic Assistant Chatbot with pattern-matching
6	Extended chatbot responses: greetings, CAT schedule, registration, NLP topics
7	Added --demo flag for non-interactive screenshot-friendly demonstration
8	Tested chatbot with 10 different queries and documented pattern-matching logic

5.3 Technologies Used

Technology	Purpose	Source
NLTK	Complete NLP pipeline — tokenization, POS, stopwords	NLTK
spaCy	Dependency parsing and semantic similarity (Stanford)	spaCy
Python stdlib	time, sys, random — chatbot interaction logic	Python stdlib
collections.Counter	N-gram frequency analysis	Python stdlib
math	Log-probability computation for language models	Python stdlib

5.4 Code Snippets

NLTK	Complete NLP pipeline — tokenization, POS, stop words	pip install nltk
spaCy	Dependency parsing and semantic similarity (bonus)	pip install spacy
Python stdlib	time, sys, random — chatbot interaction loop	Built into Python
collections	Counter for n-gram frequency analysis	Python stdlib
math	Log-probability computation for language models	Python stdlib

5.4 Code Snippets

// Complete NLP Pipeline — week5/nlp_pipeline_complete.py

```
# — NLTK imports —
try:
    import nltk
    from nltk.tokenize import word_tokenize, sent_tokenize
    from nltk.corpus import stopwords
    from nltk.stem import PorterStemmer, WordNetLemmatizer
    from nltk import pos_tag
except ImportError:
    print("ERROR: NLTK not installed. Run: pip install nltk")
    sys.exit(1)

# — Download required NLTK data —
for pkg in ["punkt", "punkt_tab", "stopwords", "wordnet", "averaged_perceptron_tagger",
            "averaged_perceptron_tagger_eng"]:
    try:
        nltk.data.find(f"tokenizers/{pkg}")
    except LookupError:
        nltk.download(pkg, quiet=True)

# — spaCy import (optional – graceful fallback) —
try:
    import spacy
    _nlp = spacy.load("en_core_web_sm")
    SPACY_AVAILABLE = True
except Exception:
    SPACY_AVAILABLE = False

# — Tools —
stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()
STOP_WORDS = set(stopwords.words("english"))

DIVIDER = "=" * 70
SEPARATOR = "-" * 70
```

Fig 5.1: Complete integrated NLP pipeline — NLTK imports, graceful spaCy fallback, full tool initialisation

5.5 Practical Outputs

```
Bot : Hi there! I am the B114133 NLP Academic Assistant. How can I help you?

You : What is tokenization?
Bot : Tokenization splits text into individual words or sentences. Use nltk.word_tokenize() for words, nltk.sent_tokenize() for sentences.

You : Explain stemming
Bot : Stemming reduces a word to its base form by removing suffixes. Example: 'running' -> 'run'. Use PorterStemmer from NLTK.

You : What is lemmatization?
Bot : Lemmatization returns the dictionary form of a word using grammar rules. More accurate than stemming. Use WordNetLemmatizer from NLTK.

You : Explain POS tagging
Bot : POS tagging assigns grammatical roles: NN=Noun, VB=Verb, JJ=Adjective, DT=Determiner. Use nltk.pos_tag(tokens).

You : What is an HMM?
Bot : HMM (Hidden Markov Model) is a sequence labeler. In this project it labels each word as CROP, DISEASE, SYMPTOM, LOCATION, ACTION, or O.

You : What is the Viterbi algorithm?
Bot : The Viterbi algorithm uses dynamic programming to find the most likely label sequence in  $O(N^2 * T)$  time.

You : Explain n-grams
Bot : N-grams are sequences of N words. Bigrams=2 words, Trigrams=3 words. Used to model language patterns and predict next words.

You : What is dependency parsing?
Bot : Dependency parsing shows how words in a sentence relate grammatically. ROOT=main verb, nsubj=subject, dobj=object.

You : What is semantic similarity?
Bot : Semantic similarity measures how close two sentences are in meaning. Range: 0.0 (unrelated) to 1.0 (identical). Use nltk.util.fuzzy_ratio.
```

Python/Library Help	Guides installation of NLTK, spaCy
---------------------	------------------------------------

Fig 5.2: Student Academic Assistant chatbot demo — answering NLP queries on tokenization, HMM, Viterbi, n-grams, dependency parsing, semantic similarity

5.6 Learning Outcomes

- **Pipeline Integration:** Chaining tokenization → stopwords removal → stemming → lemmatization → POS tagging shows how each stage enriches text representation for downstream tasks.
- **Rule-based vs ML Chatbots:** Rule-based chatbots match keywords deterministically — fast and predictable, but brittle on unseen inputs. ML-based chatbots (e.g., ChatGPT) use trained neural models — flexible but require large training data.
- **NLP Workflow Understanding:** The 8-step workflow applies to virtually every real-world NLP system, from search engines to voice assistants.

Reflection on Challenges

- Covering all keyword variants (e.g., "cat", "cat1", "cat 1", "test", "exam") for reliable pattern matching.
- Handling misspelled inputs gracefully without pattern matching completely failing.
- Balancing simple rule-based logic with sufficiently varied chatbot response options.

5.7 GitHub Commit

Commit 69b3aef

Joseph Muturi committed 7 minutes ago

Add weeks_demo.py: standalone Week 5 output demo for screenshot (pipeline + chatbot, no external dependencies)

main

GITHUB : <https://github.com/Josemuturi/NLP>

Commit 5 (69b3aef): Complete integrated NLP pipeline script, standalone Week 5 output demo (pipeline + chatbot, no external dependencies)

